C291 - Spring 26 - System Programming with C & Unix Assignment 9

Question 1: Invoice Printing (20 marks)

An invoice consists of a list of line items, a delivery address. A line item consists of a product name and quantity. A product includes a name and unit price. An address includes a street address, city, state and zip. Use structures to represent invoice, line item, product and address. Write a program in C to print an invoice. Use the provided example to initialize the variables. No user input is required. Your program must implement the following functions:

- *Address createAddress(char * street, char * city, char * state, char * zip)*

  Returns an address created with the input arguments

- *Product createProducts(char * name, double price)*

  Returns a product created with the input arguments

- *Item createLineItem(Product prod, int quantity)*

  Returns a line item created with the input arguments

- *Invoice createInvoice(Items * items, Address address)*

  Returns an invoice created with the input arguments

- *void printInvoice(Invoice invoice)*

  Prints the invoice received as input argument. The format does not need to exactly match Figure 1.

```
                I N V O I C E

Sam's Small Appliances
100 Main Street
Anytown, CA  98765

Description                    Price Qty  Total
Toaster                        29.95   3   89.85
Hair dryer                     24.95   1   24.95
Car vacuum                     19.99   2   39.98

AMOUNT DUE:  $154.78
```

Figure 1: Sample invoice

Question 2: Vending Machine (30 marks)

Write a program that simulates a vending machine. The program should use a struct to hold the information of each item. An item should have a name, label(i.e. A1, B1, C2), price, and quantity. The vending machine should have 3 rows and 3 columns, and you can create an array of items to represent it.

The program should also have a menu displaying the items with names and labels. The menu should also display the following options for customers:

- *Insert money*
  - *Ask the customer to input how much money they would like to insert.*
- *Check current balance*
  - *Display how much money the customer has inserted.*
- *Check price*
  - *Ask the customer to input a label and display the price of the item with that label. If the label does not exist, it should display a message indicating that and continue.*
- *Purchase*
  - *Ask the customer to input a label and purchase the item for them. If the current balance the customer inserted is not enough for this item, a message to be displayed to indicate that they don't have enough money. Likewise, if the label does not exist or the item has zero quantity, a corresponding message should be displayed as well.*
- *Return change*
  - *Display a message that how much money left in the vending machine is returned to the customer.*
- *Exit*
  - *Terminate the program.*

In addition, there should also be options for the owner:

- *Add a new type of item*
  - *Ask the owner to input an item name and price to create a new type of item and add to the vending machine. Your program should assign a label for the item and the quantity can start with zero. If the vending machine has no space for a new item, a message should be displayed.*
- *Restock*
  - *Ask the owner to input a label and quantity to restock. If the label does not exist, display a message about it.*
- *Check quantity*
  - *Display the current quantity of that item with input being a label.*
- *Take down a type of item*
  - *Ask the owner to input a label and remove that item from the vending machine. If the label does not exist, display a message about it.*
- *Check total sales*
  - *Display how much money the vending machine has earned.*
- *Return to the customer menu*

Your initial menu could look something like this:

```
---------------------------------------
|Coke       |Orange Soda |Root Beer   |
|A1         |A2          |A3          |
---------------------------------------
|Water      |Grape Soda  |Coke Zero   |
|B1         |B2          |B3          |
---------------------------------------
|Cream Soda |Lemonade    |Lime Soda   |
|C1         |C2          |C3          |
---------------------------------------

[1] Insert money
[2] Check balance
[3] Check price
[4] Purchase
[5] Return change
[6] Owner menu
[7] Exit

Please select an option:
```

1-7 represents the index of options the user can input to interact with the menu.
If the user input 6 which is the owner menu, your program should go to the owner menu and display the owner options.

```
Owner Menu

[1] Add a new type of item
[2] Restock
[3] Check quantity
[4] Take down a type of item
[5] Check total sales
[6] Exit owner menu

Please select an option:
```

***Note**:*
Your program should not terminate until the user select the exit option on the customer menu. You can achieve this with a while loop. As for the owner menu, you can use a nested while loop inside the main loop as well.
Your program can start with some items already in the vending machine so the initialize menu doesn't look empty.

Here's an example on how the program should behave during runtime:

```
---------------------------------------
|Coke       |Orange Soda |            |
|A1         |A2          |A3          |
---------------------------------------
|           |            |            |
|B1         |B2          |B3          |
---------------------------------------
|           |            |            |
|C1         |C2          |C3          |
---------------------------------------
[1] Insert money
[2] Check balance
[3] Check price
[4] Purchase
[5] Return change
[6] Owner menu
[7] Exit

Please select an option: 1
Amount: 5.00

Please select an option: 2
Current balance: $5.00

Please select an option: 3
Label: A1
Coke is $2.00

Please select an option: 4
Label: A1
You purchased a coke for $2.00

Please select an option: 2
Current balance: $3.00

Please select an option: 5
$3.00 is returned

Please select an option: 2
Current balance: $0.00

Please select an option: 6
Owner Menu
```

```
[1] Add a new type of item
[2] Restock
[3] Check quantity
[4] Take down a type of item
[5] Check total sales
[6] Exit owner menu

Please select an option: 1
Item name: Water
Item price: 1.00
Water is added to A3 with price of $1.00

Please select an option: 2
Label: A3
Quantity: 10
10 water is restocked

Please select an option: 3
Label: A3
The stock of water is 10

Please select an option: 4
Label: A1
Coke is taken down

Please select an option: 2
Label: A1
A1 is empty

Please select an option: 5
Total sales: $2.00

Please select an option: 6

----------------------------------------
|           |Orange Soda |Water       |
|A1         |A2          |A3          |
----------------------------------------
|           |            |            |
|B1         |B2          |B3          |
----------------------------------------
|           |            |            |
|C1         |C2          |C3          |
----------------------------------------
```

```
[1] Insert money
[2] Check balance
[3] Check price
[4] Purchase
[5] Return change
[6] Owner menu
[7] Exit

Please select an option: 7
Program exited
```

Question 3: Array Struct (30 marks)

We have learnt that there is no array.length property in C and we need to pass the size/length of the array to every function that receives this array as argument. To fix this problem, we will create a structure containing an array (actually, a pointer to an array) and its length as members. We will restrict the array type to int only.

Here's the definition of the struct:

```
typedef struct Array Array;
struct Array {
     int* data;
     size_t length;
};
```

data is a pointer pointing to the allocated memory and *length* is a value representing how many integers are there in the array.

This way, if we are going to loop through the array somewhere else in our program, we don't need to remember how long the array is. We can write a loop like how we are initializing the array elements in order to loop through the array.

So, if we define `Array* arr` as the array, we can loop through it using the following code snippet

```
for (int i = 0; i < arr->length; i++) {
     printf("%d ", arr->data[i]);
}
```

Implement the following functions to create and maintain this structure.

- `Array* new_array(size_t len)`
    - This function should allocate memory for both the array struct, and the data based on the length given as the parameter. In addition, it should also initialize all the elements as zero. In the end, it should return the pointer to our array struct.

    ***Note**: It would be easier to think this function as the **new** keyword in C++, Java and C# if you are familiar with those languages.*

- `int array_get(Array* arr, int index)`
    - This function should return the item at the given index. Note that you need to pass the pointer to the array struct as a parameter.

- `void array_set(Array* arr, int index, int value)`
    - This function should set a value to the given index in the array struct.

- `void delete_array(Array* arr)`
    - This function should free the memory allocated for the data and the array struct by the *new_array* function.

Here are some examples:

```
Array* arr = new_array(5);

array_set(arr, 0, 2);
array_set(arr, 1, 3);
array_set(arr, 2, 5);
array_set(arr, 3, 7);
array_set(arr, 4, 11);

for (int i = 0; i < arr->length; i++) {
     printf("%d ", array_get(arr, i));
}

delete_array(arr);
```

The code block above will print: 2 3 5 7 11

Extra Credits (15 marks)

After doing problem 3, think about how we can make our array struct take any type of items. For example, we can put integer type value, double type value and even custom struct type values into the same array.

Create a new struct named *ArrayAny,* or *ArrayNonGeneric* or anything you like, then implement the four functions: *new_array, array_get, array_set* and *delete_array* for this new array struct.

Note: The code for this problem should be in a separate file.

Question 4: Find duplicates (20 marks)

Consider an array of size N+1 integers. The integers are in random order, but each of the integers are between 1 and N (inclusive). In addition, each number appears only once in the array, except for one number, which occurs twice. You need to find the number that occurs twice in this array **using the bitwise XOR operator**.

Algorithm:

1. Compute the bitwise XOR of all the array indices
2. Compute the bitwise XOR of all the array elements
3. Compute the bitwise XOR of the result of 1 and 2 (this will be the duplicate number)

Prompt the user to provide an array of (N+1) elements such that

1. it includes all the numbers from 1 to N and
2. one of the numbers is repeated.

Example 1:

Input:

Enter the size of the array: 4

Enter the elements of the array: 1, 2, 2, 3

In the above example, N = 3, so the size of the array is N+1 = 3+1 = 4, and elements should be between 1 and 3 only with only one element, i.e. 2 occurring twice.

Output:
2

Since 2 occurs twice in the array, the output is 2.

Example 2:

Input:

Enter the size of the array: 8

Enter the elements of the array: 1, 2, 3, 4, 5, 6, 7, 7

In the above example, N = 7, so the size of an array is N+1 = 7+1 = 8, and elements should be between 1 and 7 only with only one element, i.e. 7 occurring twice.

Output:
7

Since 7 occurs twice in the array, the output is 7.

Constraints:

Elements in the array should include every number between 1 and N only, where N+1 is the size of the array. Only one element in the array would occur twice.